

Evidencia 2. Actividad Integradora de Gráficas

Carlos Delgado Contreras A01712819

Modelación de sistemas multiagentes con gráficas computacionales (Gpo 101)

30 de Agosto de 2026

Índice de Contenido

1. Ficha del juego.....	3
2. El play loop.....	4
Objetivo principal.....	4
Acciones para avanzar.....	4
Condiciones de fallo.....	4
Flujo post-resultado.....	4
3. Decisiones técnicas.....	5
Problema técnico 1.....	5
Problema técnico 2.....	6
Problema técnico 3.....	7
4. Qué reutilizaste y qué apoyos usaste.....	10
Scripts de clases y laboratorios.....	10
Código desarrollado desde cero.....	10
Declaración sobre uso de Inteligencia Artificial.....	10
5. Créditos de assets.....	12
6. Playtest.....	16
Información del tester.....	16
Observaciones clave (sin intervención).....	16
Cambios realizados a partir del playtest.....	16
7. Reflexión.....	17
8. Enlaces.....	18

1. Ficha del juego

- **Nombre del juego:** AirZ: Zombie Hunter
- **Descripción:** Auxilia y defiende desde las alturas a los supervivientes del ataque zombie. Lanza el paquete de ayuda y defiende a los superviviente de la horda zombie desde tu avión de combate
- **Controles:**
 - Mouse/TrackPad: Mover cámara
 - Click izquierdo/botón x: Disparar
 - Click derecho/ boton z: aumentar zoom
 - Barra Espaciadora: Cambiar de modo
- **Plataforma:** PC & WebGL

2. El play loop

Objetivo principal

El jugador debe de lanzar cierta cantidad de paquetes de ayuda y defender su posición utilizando sus armas ante los ataques zombie. Su objetivo es eliminar a toda la horda de zombies antes de que destruyan el paquete de ayuda.

Acciones para avanzar

El jugador debe de lanzar el paquete de ayuda desde el avión. Una vez este toque el piso deberá de usar su arma para eliminar a las hordas zombie desde las alturas. Una vez la horda zombie sea eliminada el paquete de ayuda es recogido. Se debe de repetir esta secuencia cierta cantidad de veces para pasar el nivel.

Condiciones de fallo

Los zombies y las balas de tu arma causan daño al paquete. Si un zombie llega a la posición del paquete le resta daño. Si la salud del paquete llega a cero este quedará destruido y se te restará una vida. Si tus vidas llegan a cero habrás perdido la partida.

Flujo post–resultado

- **Al ganar:** Se muestra una pantalla de victoria. Donde puedes salir al menú o volver a jugar.
- **Al perder:** Se muestra una pantalla de derrota. Donde puedes salir al menú o volver a jugar.

3. Decisiones técnicas

Problema técnico 1

- **Contexto y razón (El porqué):** El movimiento de la cámara del jugador simula una torreta de combate. Esta debe de poder **rotar** tanto en el eje X como en el eje Y a libertad. El problema es que al mover los dos ejes al mismo tiempo sufría un **gimbal lock**. La cámara del jugador se movía de forma errática y sin sentido.
- **Implementación:**

La solución fue añadir una capa más de control de rotación a la jerarquía. Me refiero a que se añade un GameObject vacío como padre de la cámara el cual gestionará el eje de rotación x (yaw) y la cámara el eje de rotación y (el pitch). Al separar las rotaciones de los ejes se eliminó el gimbal lock. A continuación se presenta el código en el GameObject intermedio llamado GunshipTurret.

CSharp

```
// Código en Script de GunshipTurret//  
//1. Leemos los inputs del mouse  
float mouseRawX = Input.GetAxis("Mouse X") * sensibilidad * delta;  
float mouseRawY = Input.GetAxis("Mouse Y") * sensibilidad * delta;  
//print("Current Y rotation: " + mouseRawY);  
//print("Current X rotation: " + mouseRawX);  
  
//2. Los acumulamos y limitamos  
currentRotationX -= mouseRawY;  
currentRotationX = Mathf.Clamp(currentRotationX,minY,maxY);  
  
currentRotationY += mouseRawX;  
currentRotationY = Mathf.Clamp(currentRotationY,minX,maxX);  
  
//3. Aplicar la rotacion en coordenadas locales  
transform.localRotation = Quaternion.Euler(0f, currentRotationY,  
0f);  
//print("Local rotation: " + transform.localRotation);  
  
// La cámara rota sobre el eje X del pivot (arriba/abajo)  
if (PlayerCamara != null)  
{
```

```

        PlayerCamara.localRotation = Quaternion.Euler(currentRotationX,
0f, 0f);
        //print("Camera Rotation: " + PlayerCamara.localRotation);
    }

```

Problema técnico 2

- **Contexto y razón (El porqué):** El arma del jugador busca simular ser una ametralladora Gatling de altísima cadencia. El problema es que instanciar y eliminar los prefabs de balas es demasiado costoso en rendimiento.
- **Implementación:** La solución fue generar un Object Pool de Prefabs para las balas. Este Pool de objetos instala al inicio de la partida cierta cantidad de instancia de balas y las pone en estado inactivo. Al disparar, se pide activar una bala en cierta posición con cierta rotación. El Pool despierta a una bala, o crea una en caso de ya no tener, y la devuelve para su configuración. Al colisionar la bala, se desactiva y vuelve al pool para ser usada de nuevo. A continuación se muestra el código de Pool de Balas. Este patrón de diseño también se usó en los enemigos y en las partículas de Ground Hit.

CSharp

```

//Codigo en Script de GunshipTurret//
void initializeObjectPooling(){
    for(int i = 0; i< InitialPoolSize; i++){
        bullet nuevaBala = Instantiate(bulletPrefab,Vector3.zero,
Quaternion.identity);
        nuevaBala.transform.SetParent(BulletContainer.transform);
        nuevaBala.deactivate();
        nuevaBala.onBulletDeactivated += RegresarBalaAlPool;
        //La registramos en el Object Pooling :)
        SleepingBulletPool.Enqueue(nuevaBala);
    }
}

if(Input.GetButton("Fire1")){
    if(currentTime <= 0 && currentTemp < maxTemp && !overload){
        num += 1;
        bullet newBullet;
        Quaternion thermalBloom = GetThermalBloomRotation();
        if(SleepingBulletPool.Count > 0){
            newBullet = SleepingBulletPool.Dequeue();

```

```

newBullet.activate(SpawnPoint.transform.position,
thermalBloom);

    } else{
        //AutoEscalamiento
        newBullet = Instantiate(bulletPrefab,Vector3.zero,
Quaternion.identity);

newBullet.transform.SetParent(BulletContainer.transform);

//print("Spawn point en: " +
SpawnPoint.transform.position);
        newBullet.activate(SpawnPoint.transform.position,
thermalBloom);

    }

```

Problema técnico 3

- **Contexto y razón (El porqué):** Para saber si el jugador ya eliminó a toda la horda de zombies, el **Spawn Manager** debe de detectar que su Stock está vacío y que ya no existen enemigos en la partida. Una vez sepa esto debe de mandar una señal al Game Master para aumentar el puntaje del jugador y reiniciar las mecánicas. El problema es que el zombie debe de avisar cuando “muere” para que el spawn Manager se entere.
- **Implementación:** Debido a que los Enemigos contiene una arquitectura de Object Pooling. Implemente dos estructura de datos:
 - Queue: Para los zombies dormidos
 - HashSet: Para los zombies activos

Cuando un enemigo es despertado pasa del Queue al HashSet. Para que todo zombie pueda comunicarse con el Enemy Pool hicimos de este ultimo lo que se conoce como un **Autoload** (instance). Un script accesible por cualquiera en cualquier momento. De esta forma los zombies pueden llamar a una función para avisar al Enemy Pool que han muerto y actualizar el HashSet. De esta manera el Spawn Manager puede consultar su propio stock y el Count del HashSet. Si ambos estan en zero y el paquete no mando la señal de Destroyed, entonces el jugador pudo defender correctamente el paquete. A continuación se presenta codigo clave de este sistema.

CSharp

```

//Codigo en Script de EnemyPool //
public class EnemyPoolManager : MonoBehaviour

```

```

{
    public static EnemyPoolManager Instance { get; private set; }

    [Header("Pool Config")]
    public int EnemyPoolSize = 50;
    public Zombie zombiePrefab;
    // Zombies inactivos actualmente en la isla
    private Queue<Zombie> EnemyPoolQueue = new Queue<Zombie>();
    // Zombies activos actualmente en la isla, se supone que HashSet es
    super rapido
    private HashSet<Zombie> activeZombies = new HashSet<Zombie>();
    // Método para devolver al zombie a la Queue cuando muere
    public void ReturnZombieToPool(Zombie zombie)
    {
        activeZombies.Remove(zombie);
        zombie.gameObject.SetActive(false);
        EnemyPoolQueue.Enqueue(zombie);
        print("Zombie returned, size of hash: " + activeZombies.Count);
    }

    public int getActiveZombiesCount(){
        return activeZombies.Count;
    }

    // Funcion en Script de Zombie //

    // Método de desactivación para devolver al Pool
    public void Despawn()
    {
        if (agente != null && agente.isOnNavMesh)
        {
            agente.ResetPath();
            agente.enabled = false; // Desactivar agente evita que procese
en background
        }

        //Caso A: Tenemos referencia
        if (enemyPoolManager != null)
        {
            EnemyPoolManager.Instance.ReturnZombieToPool(this);
        } //Caso B: No tenemos ref, la buscamos
        else if (EnemyPoolManager.Instance != null)
        {

```



```
        EnemyPoolManager.Instance.ReturnZombieToPool(this);
    } //Caso C: No tenemos ref y no encontramos, nos desactivamos
    else
    {
        Debug.LogError("Perdida de Pool, enemigo no encontro Pool a  
regresar");
        gameObject.SetActive(false);
    }
}
```

4. Qué reutilizaste y qué apoyos usaste

Scripts de clases y laboratorios

Me apoyé bastante en la arquitectura de timers vistos en los laboratorios para muchos sistemas. La mayoría de los sistemas requieren de una variable/dato incrementado o decrementado a través del tiempo. El uso de delta del motor de Unity fue clave para que la mayoría de los sistemas pudieran funcionar con éxito.

Código desarrollado desde cero

A continuación se enumeran los sistemas diseñados y codificados por mi persona para realizar este proyecto

1. Sistema de Torreta (Aim/disparo/Estados)
2. Spawn Manager
3. Object Pooling en Balas, Zombies y Partículas
4. Sistema de Órbita de Avión
5. Sistema de temperatura de Arma
6. Sistema de Bloom de arma
7. Sistema de pathfinding de zombies
8. Mini motor de Reynolds en zombies
9. Sistema de Zoom de cámara.

Declaración sobre uso de Inteligencia Artificial

Se utilizó Inteligencia artificial en el desarrollo de este proyecto tanto en forma de aprendizaje y guía como codificación directa en el proyecto.

- **Herramienta utilizada:** Claude & Gemini
- **Alcance del apoyo:** Las herramientas de Inteligencia artificial me ayudaron en el aprendizaje y guía para la elaboración de varios sistemas de este proyecto. En especial en los efectos visuales y Art Tech de Unity que me eran completamente ajenos. A continuación se listan las categorías donde la IA fungió como guía para la implementación y función de este proyecto:
 - Efectos FLIR en la cámara: Me guió en el uso de Global Volumes, la configuración del render URP y los diferentes overrides para obtener el efecto deseado
 - GUI militar: Me guió y enseñó el uso del motor de arte de Unity para generar el Crosshair, la barra de temperatura y los efectos de letras sin ningún asset.

- Bugs de Quaternions en Cámara y Vuelo orbital: Fue de gran utilidad para entender y corregir el sistema de vuelo orbital y los errores conceptuales de Quaternions en mi código. Desde cómo girar el avión hacia la dirección de la órbita hasta como inicializar la cámara viendo al centro del Terrain. Varios efectos requieren un conocimiento avanzado del motor de Quaternions por lo que fue de gran ayuda para el entendimiento.
- **Codificación directa con IA:** En el proyecto hubo funcionalidades y funciones que fueron directamente codificadas con inteligencia artificial. Varias de los efectos y sistemas requieren de funciones avanzadas de Unity y en especial entendimiento del uso de **referencias** de los GameObjects. En el código se indica en comentarios las funciones y scripts generados o modificados directamente con inteligencia artificial. A continuación se mencionan algunos de los sistemas que fueron generados con inteligencia artificial:
 - Sistema de referencias robusto: La IA generó una conexión avanzada de señales por medio de un Target Channel para conectar a los enemigos con los MapMakers.
 - Conversión de coordenadas 3D a puntos en cámara 2D: La IA generó las fórmulas para esta conversión en el sistema de Markers.
 - Bloom y Cámara Shake: Las fórmulas y las ecuaciones para el Bloom y el Cámara Shake del jugador fueron generadas con inteligencia artificial.
- Cabe recalcar que el uso de inteligencia artificial en este proyecto fue con motivo de cubrir la falta de experiencia en temas, sistemas y sintaxis avanzada de Unity en mi persona. Los sistemas centrales del juego fueron diseñados y codificados por mi bajo la asistencia de la inteligencia artificial.

5. Créditos de assets

Pack / Recurso	Autor / Creador	Enlace de descarga	Licencia
Zombie ▾	ArtStore3D	https://assetstore.unity.com/packages/3d/zombie-120120Low%20Poly	Standard Unity Asset Store EULA
Avion ▾	HESSBURG	https://assetstore.unity.com/packages/3d/airplane-120120Unity-.HESBURG.-B%2D2%20Stealth	Standard Unity Asset Store EULA

Pack / Recurso	Autor / Creador	Enlace de descarga	Licencia
Decoracion ▾	ITHAPPY	https://assetstore.unity.com/account/view-content/87690	Standard Unity Asset Store EULA
SkyBox ▾	rpgwhitelock	https://assetstore.unity.com/packages/tools/gui/sky-boxes/rpg-skyboxes-free-to-use-for-commercial-projects/100000	Standard Unity Asset Store EULA
BulletTracer ▾	FEWES	https://github.com/Fewes/	MIT License

Pack / Recurso	Autor / Creador	Enlace de descarga	Licencia
		Volume tricTracer	
AirplaneSound ▾	daliacoss	https://free-sound.org/people/daliacoss/sounds/414483/	Attribution 3.0
GatlingSound ▾	SoundFX.studio	https://free-sound.org/people/SoundFX.studio/sounds/474125/	Attribution NonCommercial 4.0
HitSound ▾	toxicwafflezz	https://free-sound.org/people/toxicwafflezz/sounds/150838/	Attribution 3.0
Gatling Asset ▾	IDALGAME	https://assetstore.unity.com/acco	Standard Unity Asset

Pack / Recurso	Autor / Creador	Enlace de descarga	Licencia
		unt/assets#:~:text=CUSTOMIZE d-,Weather, -%2D%20Low%20policy	Store EULA

6. Playtest

Información del tester

- **Jugador:** Jugadores amantes de los shooters realistas, los juegos de simulación y las experiencias de estrategia. Compañeros con cierta experiencia en videojuegos.

Observaciones clave (sin intervención)

El playTest lo realizó mi hermano menor y pude observar varias cosas. El primer problema es que cuando mi hermano lograba defender un paquete, no sabía cómo o qué debía de hacer para lanzar el siguiente paquete. Además me comentó que la meta final del juego era un poco confusa.

Cambios realizados a partir del playtest

A partir de esta retroalimentación se realizó las siguientes implementaciones:

1. Sección de GUI con tu progreso en la partida. Se muestra cuántos paquetes haz podido defender además de tus puntos de vida.
2. Cuando un paquete es defendido, automáticamente el jugador es puesto en modo AUXILIARY pudiendo lanzar el siguiente paquete sin tener que cambiar de modo.

7. Reflexión

- **Aprendizajes clave:** Este proyecto me ha servido en gran medida para entender y acostumbrarme a la dinámica de elementos, sistemas y programación de Unity.
- **Mayor reto:** Los temas que más me costó entender fueron los Quaternions y el sistema de referencias de Unity.
- **Siguiente iteración:** Si tuviera una semana mas de desarrollo estos serían las nuevas funcionalidades:
 - Barra de vida en Package
 - Partícula de Fogonazo en Gatling
 - Rotación de Gatling al disparar
 - Mejora de Assets en GUI
 - Sonidos de Comandos y órdenes para recalcar eventos importantes en la partida
 - Sistema de Puntaje

8. Enlaces

- **Video de demostración (Gameplay / Showcase):**
https://drive.google.com/file/d/1eUebc16TfF7nX30q_K0vsSO-6xLFwIVC/view?usp=sharing
- **Repositorio de código fuente (GitHub / GitLab):**
<https://github.com/Citybanamex16/AirCraftAntiZombies.git>